Monolith vs Microservices-Inspired Architecture (FastAPI)

1. Before: Monolithic Foundation

Originally, the application followed a traditional "grouped-by-type" structure. While this is straightforward for initial prototypes, it quickly becomes difficult to maintain as the project grows.

Monolithic Folder Structure

```
backend/app/
├── routers/              # All API endpoints for all domains
│   ├── auth.py
│   ├── patients.py
│   ├── appointments.py
│   └── ...
├── schemas/              # All Pydantic models in one place
│   ├── auth.py
│   ├── patient.py
│   └── ...
├── models.py             # Giant file containing ALL database tables
├── crud.py               # Giant file containing ALL DB operations
├── config.py             # Global configuration
├── database.py           # Global database connection
└── security.py           # Global security utilities
```

Limitations of this approach:

- **Tight Coupling**: A change in the database schema in `models.py` might break unrelated code in another part of the system.

- **Cognitive Overload**: Developers must search across multiple directories to understand a single feature (e.g., "Patients").
- **Scaling Hurdles**: You cannot easily scale the "Auth" logic independently from the "Inventory" logic.

---

2. After: Refactored Modular Architecture

The transition toward a **Modular Monolith** (Microservices-inspired) reorganizes the code by **Business Domain** rather than code type.

Modular Folder Structure

```
backend/app/
├── core/                    # Horizontal/Cross-cutting concerns
│   ├── config.py            # App-wide settings
│   ├── database.py          # Database connection pool
│   ├── security.py          # Auth & Cryptography
│   └── logging_config.py    # Structured logging
├── modules/                 # Vertical / Independent feature slices
│   ├── auth/                # Fully encapsulated Auth domain
│   │   ├── router.py        # HTTP interface (Thin Controller)
│   │   ├── services.py      # Business logic & Async tasks
│   │   ├── models.py        # Auth-specific tables
│   │   ├── schemas.py       # Auth-specific shapes
│   │   └── crud.py          # Auth-specific DB queries
│   ├── patients/            # Fully encapsulated Patients domain
│   │   └── ...
│   └── ...
├── common/                  # Reusable cross-domain utils
└── main.py                  # The entry-point that mounts all modules
```

Advantages:

- **Domain Encapsulation**: Everything related to "Patients" lives in one folder.

- **Service Layer**: Business logic is separated from HTTP routing, enabling better unit testing and reuse.
- **Async Execution**: Heavy tasks (like emails) are handed off to background workers effortlessly.

---

3. Key Differences

Feature	Monolithic	Microservices-Inspired
Organization	By File Type (Routers, Models)	By Business Domain (Auth, Patients)
Encapsulation	Low (Everything shared globally)	High (Domains are self-contained)
Database	Centralized `models.py`	Isolated `models.py` per module
Logic Dept	Often inlined in routers	Dedicated Service Layer (`services.py`)
Tasks	Synchronous/Blocking	Asynchronous `BackgroundTasks`
Scaling	Scale the entire app	Scale vertical modules independently

---

4. Limitations & Roadmap

While the current architecture is "microservices-inspired," it is a **Modular Monolith**, not a full distributed microservices system.

Why it is not full microservices yet:

1. **Shared Runtime**: All modules still run in the same Python process. If one module crashes the interpreter, the whole app goes down.
2. **Shared Database**: Currently, all modules use the same SQLite/Postgres database. True microservices usually have a private database per service.
3. **Synchronous Communication**: Modules communicate via direct Python imports. In full microservices, they would use REST API calls or Message Queues (RabbitMQ/Kafka).

Future Roadmap to Full Microservices:

- **Containerization**: Wrapping each `app/modules/` folder into its own Docker container.
- **Event Bus**: Introducing Redis or RabbitMQ for inter-module communication.
- **API Gateway**: Adding a gateway to route traffic to independent services.
